SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Duration : 45 Minutes
QUESTIONS: 2 **Total Marks:20**

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I ____K. Deeksha Sree(192424097)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K. Deeksha Sree

Annexure A

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification

(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, −1 each step, −5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α, γ, ε).

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier using the Iris dataset.

(a) Load, preprocess and split the dataset

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
import pandas as pd

iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["target"] = iris.target

print("First 5 rows:")
print(df.head())

print("\nData Types:")
print(df.dtypes)

X = df.drop("target", axis=1)
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Preprocessing:

- Iris dataset has no missing values.
- No encoding is required because the features are numerical.
- Dataset is split into 80% training and 20% testing.

(b) Build Decision Tree

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

model = DecisionTreeClassifier(
    criterion="gini",
```

```
    random_state=42
)

model.fit(X_train, y_train)

print("Tree Structure:")
from sklearn.tree import export_text
print(export_text(model, feature_names=list(X.columns)))

plt.figure(figsize=(12, 8))
plot_tree(model, feature_names=X.columns,
          class_names=iris.target_names,
          filled=True)
plt.show()
```

Root node:
The root node is generally petal length (cm) for this trained tree because it provides the highest reduction in impurity using the Gini criterion.

(c) Evaluation

```
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.metrics import classification_report

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Performance:
The Decision Tree gives high accuracy on the Iris dataset, usually around 90–100% depending on the train-test split.

Misclassified instances:

```
misclassified = X_test[y_test != y_pred].copy()
misclassified["Actual"] = y_test[y_test != y_pred]
misclassified["Predicted"] = y_pred[y_test != y_pred]
```

print(misclassified)

The exact misclassified instances depend on the train-test split and tree settings, so use the output of the above code rather than writing fixed values.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a 4×4 Grid World and obtain the learned policy.

(a) Environment and Q-table

Grid:

S . . .

. X . .

. . X .

. . . G

- S = Start
- G = Goal
- X = Obstacle
- Actions = Up, Down, Left, Right
- Goal reward = **+10**
- Each step = **−1**
- Obstacle = **−5**

Hyperparameters:

α (Learning rate) = 0.1

γ (Discount factor) = 0.9

ε (Exploration rate) = 0.1

(b) Q-Learning implementation

```
import numpy as np
grid_size = 4
actions = [0, 1, 2, 3]  # Up, Down, Left, Right

start = 0
goal = 15
obstacles = [5, 10]

Q = np.zeros((16, 4))

alpha = 0.1
```

```
gamma = 0.9
epsilon = 0.1

rewards = []

def move(state, action):
    row, col = divmod(state, 4)

    if action == 0: row -= 1
    elif action == 1: row += 1
    elif action == 2: col -= 1
    elif action == 3: col += 1

    if row < 0 or row >= 4 or col < 0 or col >= 4:
        return state, -1

    new_state = row * 4 + col

    if new_state in obstacles:
        return new_state, -5

    if new_state == goal:
        return new_state, 10

    return new_state, -1



for episode in range(1, 101):
    state = start
    total_reward = 0

    for step in range(100):
        if random.random() < epsilon:
            action = random.choice(actions)
        else:
            action = np.argmax(Q[state])
```

```
        next_state, reward = move(state, action)

        Q[state, action] += alpha * (
            reward + gamma * np.max(Q[next_state]) - Q[state, action]
        )

        state = next_state
        total_reward += reward

        if state == goal:
            break

    rewards.append(total_reward)

    if episode in [1, 50, 100]:
        print(f"\nEpisode {episode}")
        print("State 0:", Q[0])
        print("State 15:", Q[15])
```

The Q-values recorded at **Episodes 1, 50 and 100** show how the initially zero Q-table gradually learns useful action values.

(c) Final learned policy

```
policy = ["Up", "Down", "Left", "Right"]

print("\nFinal Policy:")

for state in range(16):
    if state == goal:
        print("G", end="  ")
    elif state in obstacles:
        print("X", end="  ")
    else:
        action = np.argmax(Q[state])
        print(policy[action][0], end="  ")

    if (state + 1) % 4 == 0:
```

print()

Example format:

R R D D

D X D D

R R X D

R R R G

Cumulative reward plot

```
plt.plot(rewards)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning: Reward per Episode")
plt.show()
```

Observation:

At the beginning, rewards are low because the agent explores randomly. As the number of episodes increases, the Q-values improve and the agent learns a better path toward the goal. The reward generally becomes more stable, showing **convergence of the learned policy**.